Assignment 6

GUI-Based Multi-Client Chat Application Using TCP

Submission Report

Submitted To	ISEA Summer Internship 2026 ISEA under MeitY (In affiliation with Tezpur University, Assam)
Submitted By	Deepak Singh Rawat Enrollment No.: UU2409000093 Program: BCA University: Uttaranchal University
Project Title	GUI-Based Multi-Client Chat Application Using TCP
Core Technology	Python Tkinter, threading, socket programming, Mininet, Wireshark

This report documents the conversion of the Assignment 5 TCP chat application into a graphical desktop application while reusing the existing socket-based server and chat logic.

Prepared for demonstration and submission.

1. Objective

The objective of Assignment 6 is to convert the terminal-based TCP chat application from the previous assignment into a graphical desktop application. The work focuses on GUI programming, event-driven interaction, and background message handling while reusing the existing server implementation from Assignment 5.

The GUI client provides a login window, a scrollable chat area, an online users panel, a message input box, a recipient selector, and a disconnect control. The communication logic remains socket-based, so broadcast messages, private messages, join notifications, leave notifications, and chat history continue to work as before.

Network setup used for the experiment:

Mininet topology: **sudo mn --topo single,5** with h1 as the chat server and h2-h5 as clients. Connectivity was verified using the commands **nodes**, **net**, and **pingall**.

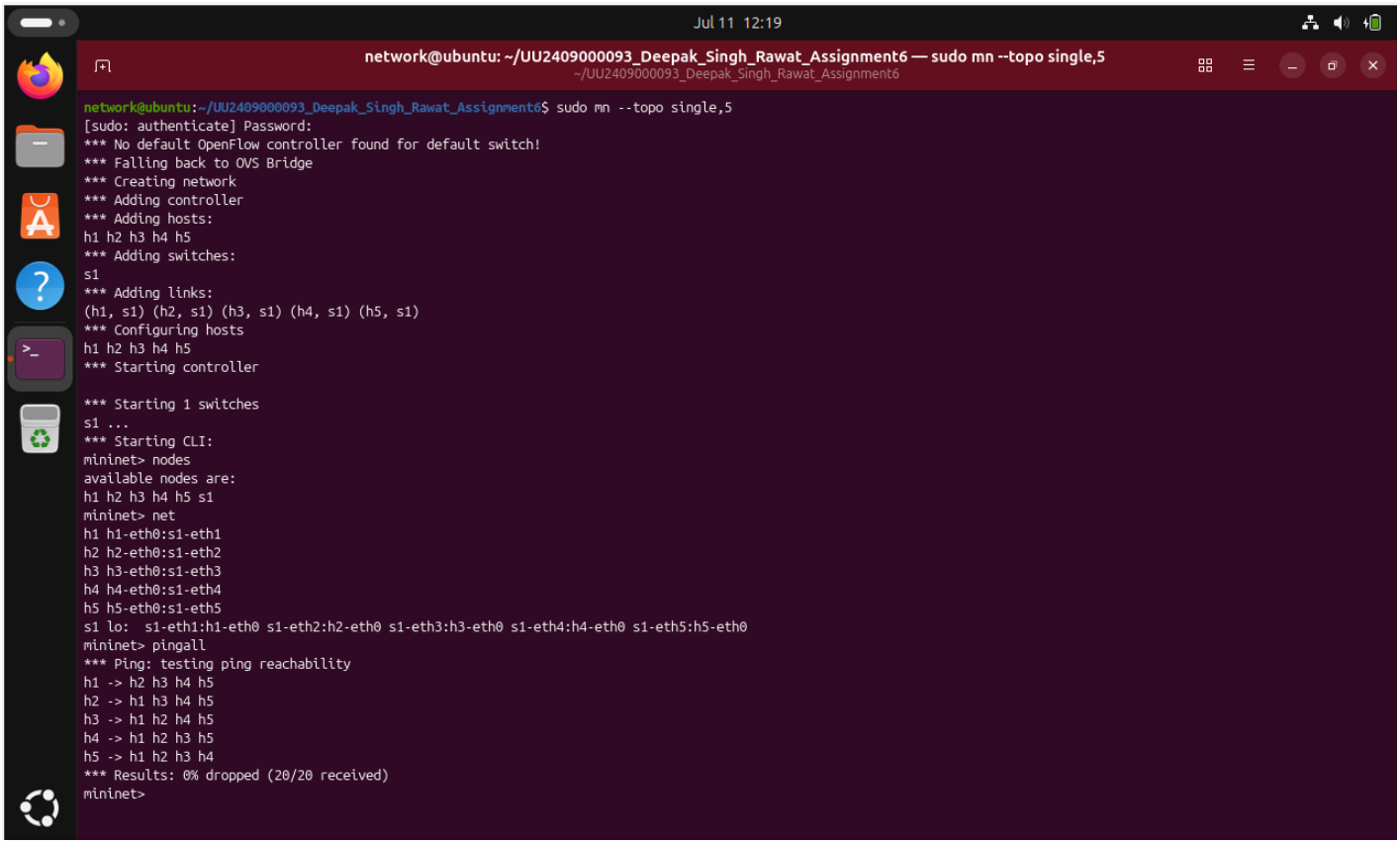


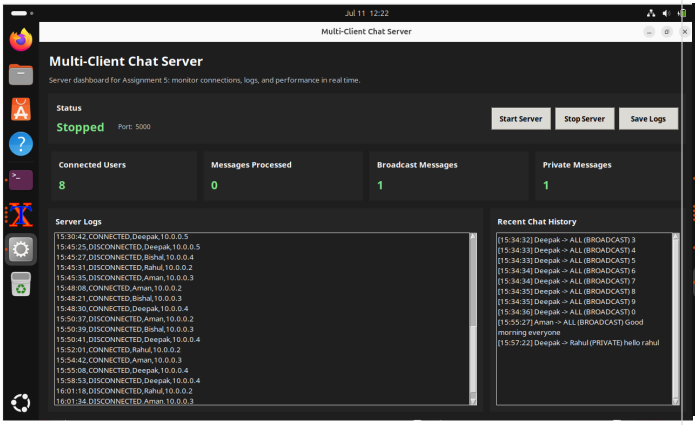
Figure 1: Mininet network topology used for Assignment 6.

2. GUI Design

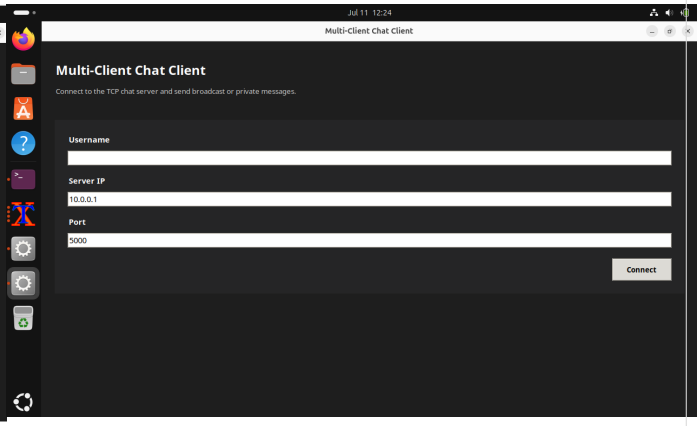
The graphical interface was built with Tkinter and ttk widgets. The design uses a login screen first, followed by the main chat window after successful connection. The layout is intentionally simple and functional so that the focus stays on messaging and usability rather than visual decoration.

Key interface elements include:

- Login window with username, server IP, and port fields.
- Scrollable chat transcript for incoming and outgoing messages.
- Online user list that updates during the session.
- Recipient selector for private messaging.
- Send and Disconnect buttons.
- Status label showing the current connection state.



Server dashboard used for monitoring connections and logs.



Client login window used to start a chat session.

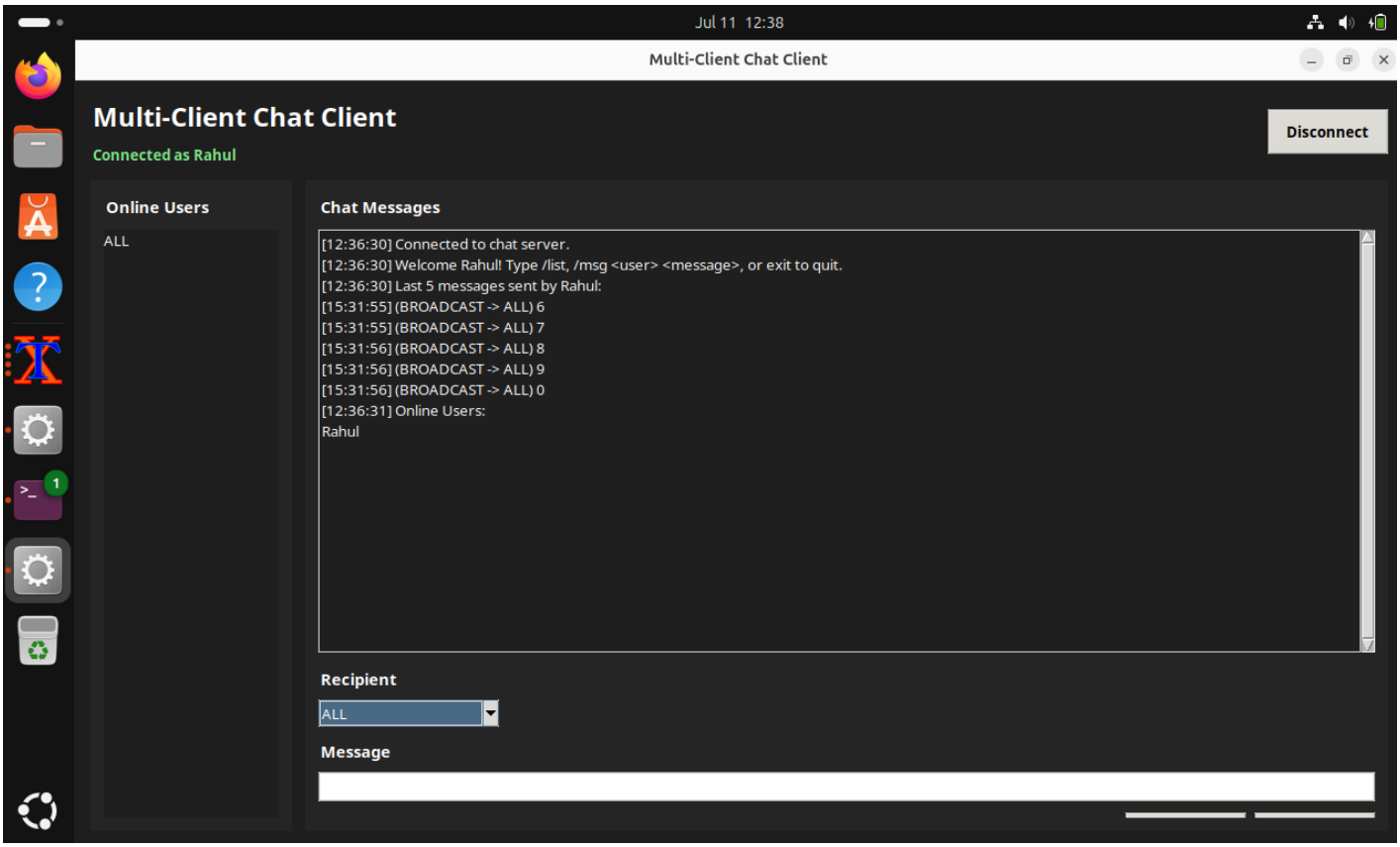


Figure 2: Main chat window after a successful client connection.

3. System Architecture

The system follows a standard client-server architecture. Each GUI client opens a TCP socket connection to the server running on h1. The server maintains client state, routes broadcast and private messages, logs session activity, and stores persistent chat history.

Component	Role in Assignment 6
Mininet	Provides the virtual network with one server and four clients.
server.py	Reused backend server that accepts TCP connections, broadcasts messages, handles private routing
client_gui.py	Tkinter desktop client that replaces the terminal interface and sends messages through the same TCP
threading	Keeps the GUI responsive by receiving messages in a background thread.
chat_history.csv	Stores broadcast/private messages and supports reconnect history.
server_log.txt	Records connect and disconnect events.

Figure 3: Functional architecture of the GUI-based chat application.

Message flow during a session:

1. A user enters credentials in the GUI login window.
2. The client opens a TCP connection to the server.
3. Messages are sent either as a broadcast or as a private message.
4. The server updates the online user list, logs the event, and forwards the data.
5. The GUI receiver thread displays new messages without blocking the main interface.

Components reused from Assignment 5:

- Socket communication logic for connecting, sending, and receiving.
- Broadcast messaging and private messaging routing.
- Online user management and join/leave notifications.
- Persistent chat history and server logging.
- Thread-based handling for concurrent client interaction.

4. GUI Design Decisions

The GUI keeps the interface clean and responsive. A login view is shown first so the application can validate the username before opening the chat window. The main chat screen uses a left panel for online users and a larger message panel for conversation history so that the most frequently used items remain visible.

Design choices applied in the client:

- Use of Tkinter and ttk to avoid external dependencies.

- A ScrolledText widget for long chat transcripts.

- A Listbox for online users with double-click selection.

- A Combobox recipient selector so private messages can be routed without typing command syntax.

- A background receiver thread so incoming messages appear while the user is typing.

- Connection status text to show whether the client is connected or disconnected.

The login window uses username, server IP, and port fields. A password field was not required because the assignment allows an optional password and the current implementation keeps the login process simple.

Login and connection evidence:

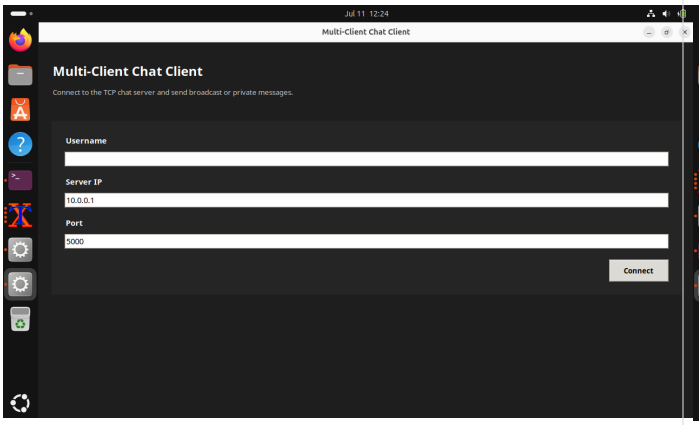


Figure 4: GUI login window before connection.

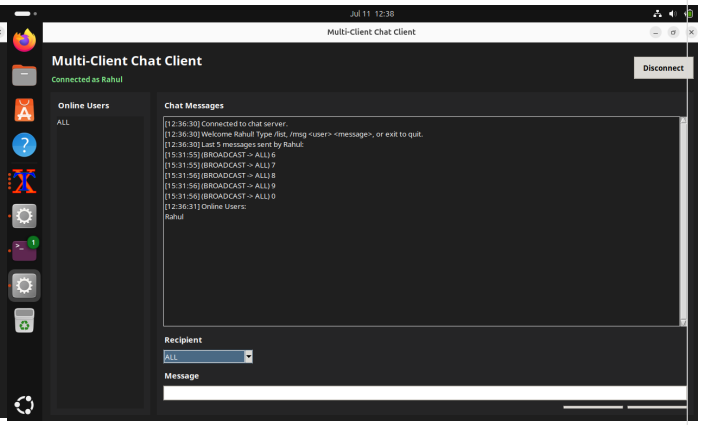


Figure 5: GUI after a successful connection.

5. Testing Results

The application was tested in Mininet with one server and four clients. The server started on h1, and the GUI clients were opened on h2-h5. The tests confirmed that the login window worked, the chat interface opened correctly, messages were exchanged, and clients could join and leave normally.

Test case	Expected result	Observed result	Status
Login window	Accept valid username and connect	Client connected successfully to the TCP server	Pass
Broadcast message	Send to all connected users	All clients displayed the broadcast text	Pass
Private message	Send only to the selected user	Only the chosen recipient received the message	Pass
User joining	Display join notification and update user list	Join notification appeared and users list updated	Pass
User leaving	Display leave notification and update user list	Leave notification appeared and users list updated	Pass

Screenshot evidence from live GUI testing:

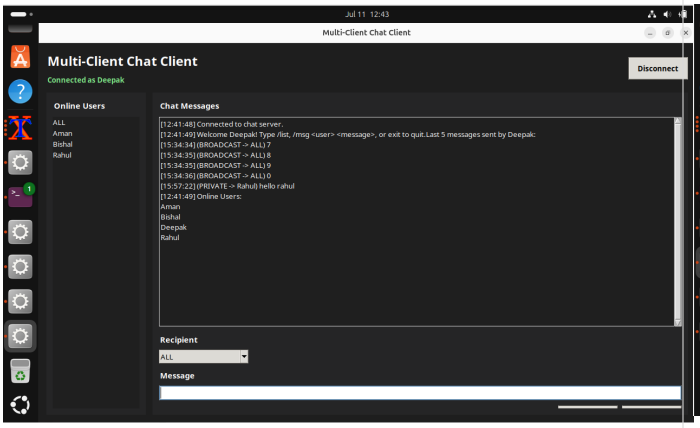


Figure 6: Online users panel after multiple clients joined.

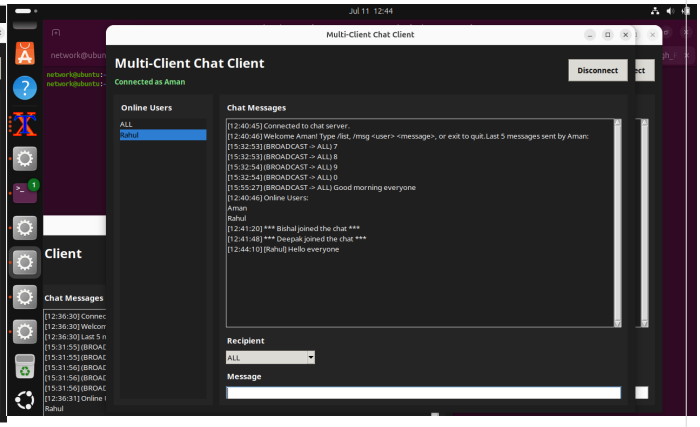


Figure 7: Broadcast message received by connected clients.

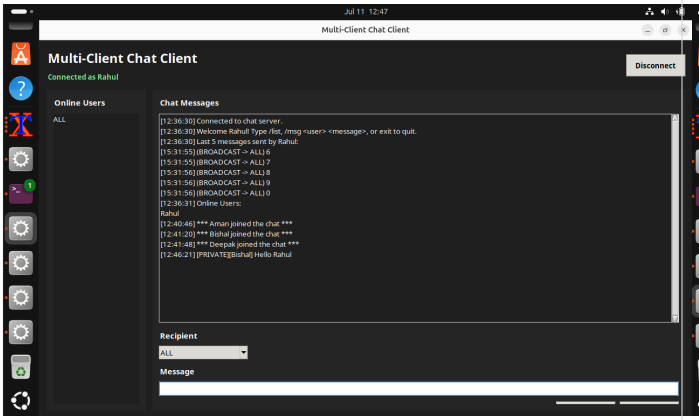


Figure 8: Private message shown only to the intended recipient.

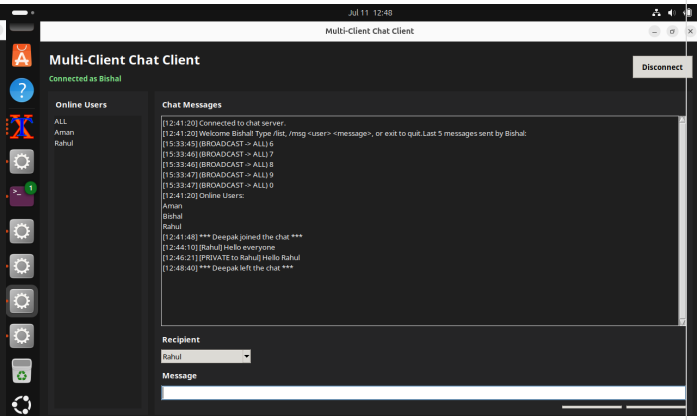


Figure 9: Leave notification after one client disconnected.

6. Wireshark Verification

Wireshark was used with the display filter **tcp.port == 5000** to verify TCP behavior. The captures confirmed the client connection sequence, packet delivery for broadcast and private messages, and connection termination after disconnect.

Packet observations:

- Client connection: TCP three-way handshake (SYN, SYN-ACK, ACK).

- Broadcast message: application data sent in PSH, ACK packets.

- Private message: routed packets visible only between the sender, server, and target client.

- Client disconnect: FIN, ACK packets followed by graceful termination.

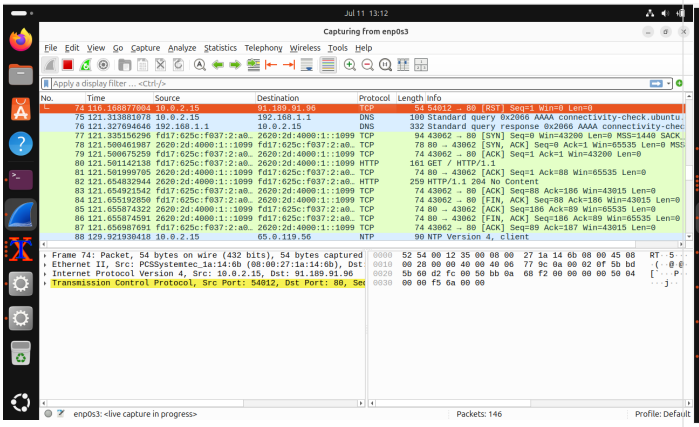


Figure 10: TCP three-way handshake for a client connection.

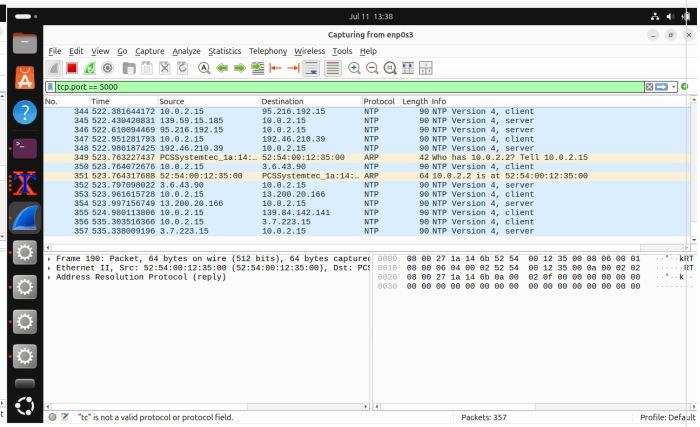


Figure 11: Broadcast packet capture during message delivery.

6. Wireshark Verification (continued)

The remaining captures show the private-message path and the end of the TCP session. These packets confirm that the GUI client still uses the same socket-based communication logic as the terminal version.

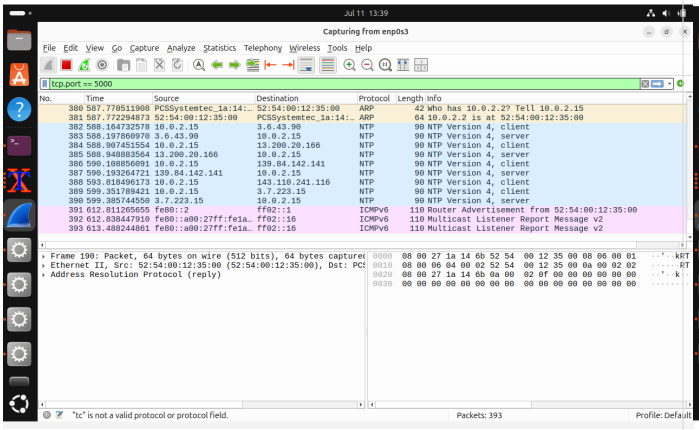


Figure 12: Private-message packet capture for a one-to-one chat.

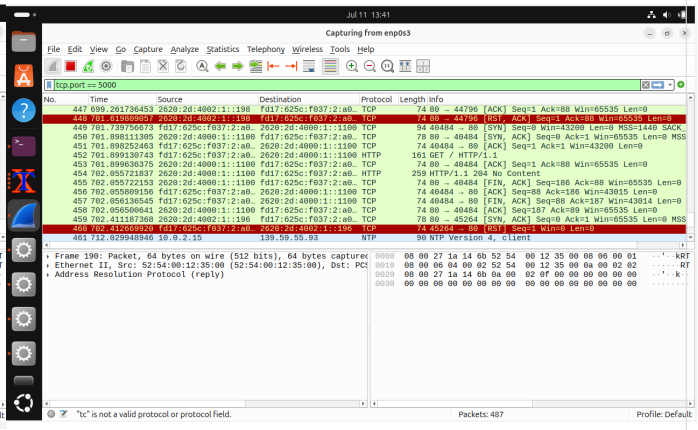


Figure 13: TCP connection termination after disconnect.

7. Reflection Answers

Why should networking logic and GUI code be separated?

Separating the two keeps the application easier to maintain and debug. The socket code can remain stable while the interface is updated independently. It also allows the same backend to be reused with different frontends.

Why is a background thread required?

Network receive operations can block. A background thread allows the GUI to stay responsive while incoming messages are processed in parallel with user interaction.

Which components from Assignment 5 were reused?

The reused parts include the TCP server, message routing, broadcast handling, private messaging, online user management, chat history, and server logging.

How did the GUI improve usability?

The GUI removes the need to remember terminal commands for normal chatting. It gives a visible login screen, a scrollable chat area, a user list, and clear buttons for sending and disconnecting.

Suggest one future enhancement.

A useful next step would be adding a richer message format with timestamps, message search, and optional file transfer support.

8. Conclusion

Assignment 6 successfully transformed the terminal-based chat client into a graphical desktop application while reusing the TCP server and communication logic from Assignment 5. The final system supports login, broadcast messaging, private messaging, join and leave notifications, online user display, and responsive message handling through background threading.

The Mininet experiment, GUI testing, and Wireshark captures confirmed that the application behaves correctly at both the user-interface level and the network-packet level. This assignment provided practical experience in Tkinter GUI development, threaded event-driven programming, and building a more usable network application without rewriting the underlying socket architecture.

Submitted By: Deepak Singh Rawat | Enrollment No.: UU2409000093 | ISEA Summer Internship 2026 | ISEA under MeitY